

Algorithmic Speedup and Implementation of the Fixed- S^z + Symmetry Construction and Symmetrized SpMV

QED exact-diagonalization library

June 2026

Abstract

We document the algorithm- and implementation-level optimizations that make a 32-site fixed- S^z plus spatial-symmetry exact-diagonalization workload (microcanonical TPQ) feasible on a single large GPU. Two hot paths are covered: (i) the *symmetry construction* (orbit-representative enumeration, sector dimensioning, and lazy / CSR-free sector data), and (ii) the *symmetrized matrix-vector product* (SpMV). The central result is replacing an orbit-CSR “expand $\rightarrow$ apply $\rightarrow$ project” SpMV—whose $\mathcal{O}(\text{full-}S^z \text{ dim})$ scatter/gather tables are streamed over PCIe every iteration—with an *on-the-fly representative* SpMV that regenerates the group action arithmetically on the device, turning the $|G|$ -fold dimension reduction into a genuine $|G|$ -fold matvec speedup while dropping the host working set from ~ 24 GiB/sector to ~ 1.2 GiB.

1 Notation and setting

For N sites at magnetization $n_\uparrow$, the fixed- S^z subspace has dimension

$$D = \binom{N}{n_\uparrow}, \quad D|_{N=32, n_\uparrow=16} = 601,080,390 \approx 6.0 \times 10^8.$$

A spatial symmetry group G (here $|G| = 8$ translations) acts on computational basis states by site permutations g . It partitions the D states into *orbits*; each orbit is labeled by its canonical *representative*

$$r_\alpha = \min_{g \in G} g(s), \quad s \in \text{orbit}(\alpha).$$

A momentum/irrep sector k keeps one symmetrized basis vector per surviving orbit,

$$|\alpha\rangle = \frac{1}{\mathcal{N}_\alpha} \sum_{g \in G} \chi_k(g)^* g |r_\alpha\rangle, \tag{1}$$

with characters $\chi_k(g)$ and normalization $\mathcal{N}_\alpha$. The sector dimension is $\approx D/|G| \approx 7.5 \times 10^7$. The goal is to build and apply H in this reduced basis *without* ever paying $\mathcal{O}(D)$ memory.

Throughout, “orbit CSR” denotes the compressed structure storing, per representative, all $|G|$ computational images $\{g |r_\alpha\rangle\}$ and their coefficients $\chi_k(g)^*$ —an $\mathcal{O}(D)$ object (≈ 14 GiB at $N=32$) plus an $\mathcal{O}(D)$ reverse index (≈ 10 GiB).

2 Symmetry construction

2.1 Orbit-representative enumeration: hash-set dedup $\rightarrow$ min-image identity

The naive enumerator scans all D states and de-duplicates representatives in a hash set (`std::unordered_set<uint64_t>` costing ~ 3.5 GB and running serially (~ 200 s; an out-of-memory source at $N=32$). We replace it with the identity

$$s \text{ is a representative} \iff s = \min_{g \in G} g(s). \quad (2)$$

Site permutations preserve popcount, so every image $g(s)$ is itself a fixed- S^z state, and the orbit minimum is hit *exactly once* as the loop sweeps the basis. No deduplication structure is needed:

Listing 1: Canonical-representative pass (per-thread, no dedup set).
include/ed/core/streaming_symmetry.h

```
1 for (size_t i = 0; i < num_basis; ++i) {
2     const uint64_t basis = basis_states_[i];
3     uint64_t rep = basis;
4     for (const auto& perm : symmetry_info.max_clique) {
5         uint64_t permuted = applyPermutation(basis, perm);
6         if (permuted < rep && lookupState(permuted) >= 0) rep = permuted;
7     }
8     if (basis == rep) thread_reps[tid].push_back(basis); // canonical
9 }
```

The per-state kernel is a bit-gather, one per group element:

Listing 2: applyPermutation — output bit i is sourced from input bit $\text{perm}[i]$.
include/ed/core/basis_utils.h

```
1 inline uint64_t applyPermutation(uint64_t basis, const std::vector<int>& perm) {
2     uint64_t result = 0;
3     for (size_t i = 0; i < perm.size(); ++i)
4         result |= ((basis >> perm[i]) & 1) << i;
5     return result;
6 }
```

Complexity. $\mathcal{O}(D|G|)$ time, $\mathcal{O}(1)$ auxiliary memory. The loop is OpenMP-parallel (each thread appends to a private vector, then a single concatenate + `std::sort` restores deterministic ascending order). Measured effect: 200 s $\rightarrow$ 6 s and elimination of the ~ 3.5 GB set.

2.2 Sector dimensions without the CSR (“Pass 1.5”)

Choosing lazy vs. eager builds and sizing solver vectors requires each sector’s dimension. Rather than *building* the orbit CSR to discover it, Pass 1.5 walks each orbit, computes only the symmetrized norm of (1), and counts survivors, storing a single integer per sector:

Listing 3: Norm-only dimension scan; thread-local scratch is discarded. `streaming_symmetry.h`

```
1 #pragma omp parallel for schedule(dynamic, 64)
2 for (size_t oi = 0; oi < num_orbits; ++oi) {
3     std::vector<uint64_t> elements; std::vector<Complex> coefficients;
4     double norm_sq = 0.0;
5     computeOrbitDataFixedSz(unique_orbit_reps_[oi], sector.phase_factors,
6                             elements, coefficients, norm_sq);
7     if (norm_sq > 1e-10) valid_count.fetch_add(1, std::memory_order_relaxed);
8 }
```

```

8 }
9 symmetrized_block_ham_sizes[sector_idx] = (int)valid_count.load();

```

`symmetrized_block_ham_sizes[k]` becomes the authoritative dimension, exposed by `getSectorDimension(k)`. This is what lets the GPU sector loop know `dim()` with *zero* orbit materialization. The orbit expansion goes through the single shared helper `compute_orbit_for_state`, so Pass 1.5, the eager build, and the CSR-free extractor of §2.4 are bit-identical.

2.3 Lazy LRU-1 materialization

The eager build materializes every sector’s orbit CSR plus reverse index (~ 190 GB across 8 sectors at $N=32$). The lazy path materializes *one* sector at a time at solve/bind time and frees the previous one (LRU-1), capping instantaneous host RSS at a single sector’s footprint (~ 24 GiB). This was the prior state of the art but still pays that $\mathcal{O}(D)$ cost while a sector is resident.

2.4 CSR-free representative data

The decisive step removes the orbit CSR from the GPU path entirely. `getRepSectorData(k)` reproduces the exact survivor set and ordering of the eager build, but stores only $\mathcal{O}(D/|G|)$ data—representatives and $1/\mathcal{N}_\alpha$ —using transient scratch for the orbit expansion:

Listing 4: CSR-free per-sector source. `streaming_symmetry.h`

```

1 std::vector<uint64_t> elems; std::vector<Complex> coeffs; double norm_sq = 0.0;
2 for (uint64_t rep : unique_orbit_reps_) {
3     computeOrbitDataFixedSz(rep, sector.phase_factors, elems, coeffs, norm_sq);
4     if (norm_sq > 1e-10) {
5         out.reps.push_back(rep);
6         out.inv_norms.push_back(1.0 / std::sqrt(norm_sq));
7     }
8 }

```

It additionally packs the $|G|$ per-sector characters $\chi_k(g)$ and the flattened group permutations—the only other quantities the device needs. Host footprint per sector: ~ 1.2 GiB (`reps` + `inv_norms`) versus ~ 24 GiB (CSR + reverse index).

Lazy host loop. With this in hand, the production sector loop (`StreamingSymmetryHandle::sector(k)`) hands out a *lazy* `SectorOperator` that (a) knows its dimension up-front (Pass 1.5), (b) builds the CSR-free record on demand for the GPU bind, and (c) materializes the orbit CSR *only* if a CPU apply is ever invoked. A pure-GPU run therefore never allocates the per-sector orbit CSR on the host.

3 The symmetrized matrix–vector product

3.1 The orbit-CSR SpMV and why it is PCIe-bound

The orbit-CSR SpMV performs *expand* $\rightarrow$ *apply* $\rightarrow$ *project*:

1. scatter the ~ 75 M reduced vector into the D -dim full basis via the orbit CSR;
2. apply H in the full basis;
3. gather back over the $|G|$ images per orbit via the orbit CSR again.

Steps 1 and 3 each *read* ≈ 13.44 GiB of orbit indices and coefficients. In lazy mode that CSR lives in host RAM and is streamed over PCIe (~ 50 GB/s) every SpMV:

$$\frac{13.44 \text{ GiB} \times 2}{50 \text{ GB/s}} \approx 0.54 \text{ s/SpMV},$$

so the $\div 8$ FLOP win is entirely consumed by host $\leftrightarrow$ device traffic, and the symmetric run is no faster than the unsymmetrized one.

3.2 On-the-fly representative SpMV: the algebraic collapse

The new path never expands. It applies H to the *single* representative of each row and reconstructs the destination index and projection phase arithmetically. For a connected computational state s' reached from r_α , let $r_\beta = \min_g g(s')$ be its destination representative. The coefficient of s' in the destination symmetrized vector is

$$\overline{\beta_{s'}} = \sum_{h: h(s')=r_\beta} \overline{\chi_k(h)}, \quad \text{projection} = \overline{\beta_{s'}} \cdot \frac{1}{\mathcal{N}_{r_\beta}}. \quad (3)$$

Derivation. In (1) the amplitude on $g r_\beta$ is $\chi_k(g) \cdot \mathcal{N}_{r_\beta}$. Writing the set $\{g : g(r_\beta) = s'\}$ as $\{h^{-1} : h(s') = r_\beta\}$ and using $\chi_k(h^{-1}) = \overline{\chi_k(h)}$ for unit-modulus characters yields (3). The reference's $|G|$ -fold orbit walk and the global $1/|G|$ weight collapse into this single representative term, so neither appears explicitly. This is implemented in one device function with no orbit table:

Listing 5: DeviceRepSymmetryBasisPolicy::index_and_projection.
include/ed/matvec/device.basis_policy.cuh

```

1 __device__ inline std::uint64_t
2 index_and_projection(std::uint64_t state, cuDoubleComplex& proj_out) const {
3     if (__popc11(state) != n_up) return kDeviceNotFound;
4     std::uint64_t rb = state; // r_b = min_g g(state)
5     for (int g = 1; g < group_size; ++g) {
6         const std::uint64_t img = apply_perm(state, g);
7         if (img < rb) rb = img;
8     }
9     const int r = ed::gpu::combinadic::rank_state(rb, n_sites, n_up);
10    const std::int32_t k = rep_index_of_rank[r];
11    if (k < 0) return kDeviceNotFound;
12    cuDoubleComplex acc = make_cuDoubleComplex(0.0, 0.0);
13    for (int h = 0; h < group_size; ++h) // sum conj(chi_k(h))
14        if (apply_perm(state, h) == rb) {
15            const cuDoubleComplex c = characters[h];
16            acc.x += cuCreal(c); acc.y -= cuCimag(c); // conjugate
17        }
18    const double s = inv_norms[k];
19    proj_out = make_cuDoubleComplex(acc.x * s, acc.y * s);
20    return (std::uint64_t)k;
21 }

```

`apply_perm` is the device twin of `applyPermutation`: the entire group action is regenerated in registers from $|G| \leq 8$ permutations and $|G|$ characters—nothing of size $\mathcal{O}(D)$ is read or streamed.

3.3 $\mathcal{O}(1)$ reverse lookup: combinadic rank and a dense rank table

Mapping $r_\beta \mapsto$ orbit index was previously a hash over all D orbit elements. It is now two $\mathcal{O}(1)$ steps.

(a) **Combinadic rank.** A fixed- S^z bitmask is converted to its colexicographic index in $[0, \binom{N}{n_\uparrow})$ by reading a Pascal triangle from `__constant__` memory (no global loads):

$$\text{rank}(s) = \sum_{b: s_b=1} \binom{b}{\#\{b' \leq b : s_{b'} = 1\}}. \quad (4)$$

Listing 6: `rank_state. include/ed/gpu/combinadic.cuh`

```

1 __device__ __forceinline__
2 int rank_state(std::uint64_t state, int n_bits, int k) {
3     int rank = 0, seen = 0;
4     #pragma unroll
5     for (int bit = 0; bit < 64; ++bit) {
6         if (bit >= n_bits || seen >= k) break;
7         if ((state >> bit) & 1ULL) { ++seen; rank += (int)binomial(bit, seen); }
8     }
9     return rank;
10 }
```

(b) **Dense rank table.** `rep_index_of_rank` maps `rank(rep)` $\rightarrow$ orbit index, length $\binom{N}{n_\uparrow}$, `int32`, with -1 for non-representatives. It is built from **reps only**—no orbit images materialized—and uploaded once per sector:

Listing 7: Reverse table built from reps only. `src/symmetry/streaming_symmetry_gpu_mirror.cu`

```

1 std::vector<std::int32_t> h_rep_index_of_rank(dim_full_sz, -1);
2 for (std::size_t i = 0; i < data.reps.size(); ++i) {
3     const std::uint64_t r = rank_combination_host(data.reps[i], n_up);
4     if (r < dim_full_sz) h_rep_index_of_rank[r] = (std::int32_t)i;
5 }
```

A destination lookup is thus a handful of constant-memory additions (`rank_state`) followed by a single `int32` global load, replacing the multi-cache-miss hash probe. Device memory for the table is $\binom{32}{16} \cdot 4 \text{ B} \approx 2.4 \text{ GiB}$ —negligible on a 40–80 GB GPU.

3.4 The kernel: one thread per representative

The kernel launches one thread per orbit representative, applies H to `reps[i]` once, and uses $1/\mathcal{N}_\alpha = \text{inv_norms}[i]$ as the source pre-phase:

Listing 8: Rep kernel core. `include/ed/matvec/term_kernels_gpu.cuh`

```

1 const double inv_norm_i = basis.inv_norms[i];
2 const cuDoubleComplex pre_phase = make_cuDoubleComplex(inv_norm_i, 0.0);
3 process_source_terms<BasisPolicy, Scalar>(
4     basis, spin_l, terms, basis.state_of(i), pre_phase, coeff_in, i, out);
```

The Hamiltonian term walk `process_source_terms` is shared verbatim with the orbit-CSR kernel (extracted into one `__device__ __forceinline__` body), so the representative path cannot diverge numerically from the validated reference. The two policies differ only in compile-time traits: `needs_orbit_walk == false` (apply to the single representative, no CSR sweep) and `has_coeff_modifier == true` (each off-diagonal emit routes through `index_and_projection`, Listing 5, for its destination and phase).

3.5 CPU hot-path lookup (fallback)

On CPU runs the reverse lookup is a 16 B/entry sorted array searched with a branch-free “monobound” binary search that prefetches both candidate probe points one level ahead, compiling the narrow step to a `cmov`:

Listing 9: SortedUint64Index::find. `include/ed/core/sorted_uint64_index.h`

```

1 while (len > 1) {
2     const std::size_t half = len / 2;
3     __builtin_prefetch(a + low + half / 2, 0, 0);
4     __builtin_prefetch(a + low + half + half / 2, 0, 0);
5     low += (a[low + half] < key) ? half : 0; // branch-free narrow
6     len -= half;
7 }

```

This removes the per-probe mispredict (the hottest op in the CPU symmetry SpMV) for $\sim 2\times$; a companion optimization pre-bakes $1/\mathcal{N}_\alpha$ so the inner loop multiplies instead of dividing per emit.

4 Net accounting

Table 1: Per-SpMV cost for one $N=32$ fixed- S^z symmetry sector.

	Orbit-CSR (lazy)	On-the-fly representative
Host→device traffic	2×13.44 GiB CSR	two ~ 75 M vectors only
Device resident	$\sim 30+$ GiB	~ 6 GiB
Reverse lookup	hash over D elements	<code>rank_state</code> + one <code>int32</code> load
Group action	materialized images	regenerated from $ G $ perms in registers
Host build / sector	~ 24 GiB CSR + index	~ 1.2 GiB <code>reps+inv_norms</code>

The orbit-CSR column is PCIe-bound and roughly matches the unsymmetrized cost; the representative column converts the $|G|$ -fold dimension reduction into a real $|G|$ -fold matvec speedup and removes the host-RSS ceiling that previously forced a 64 GB+ node. Correctness is pinned bit-for-bit against the CPU `applySymmetrized` reference across $\mathbb{Z}_N$ rings (including a $|G|=8$ complex-character fixed- S^z fixture), and the host-CSR-never-materialized invariant on the GPU path is asserted end-to-end through the production sector loop.